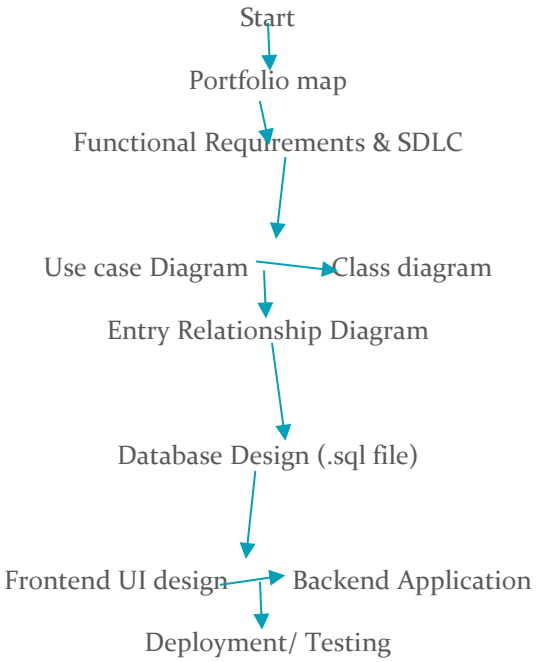


Software Design & Development

21/05/2025

Author: Rim Zino
Professor: Youness el Amrani
Year: 2024/2025
Institution: Sist

Portfolio map:



Files in the appendices.

Functional Requirements and SDLC

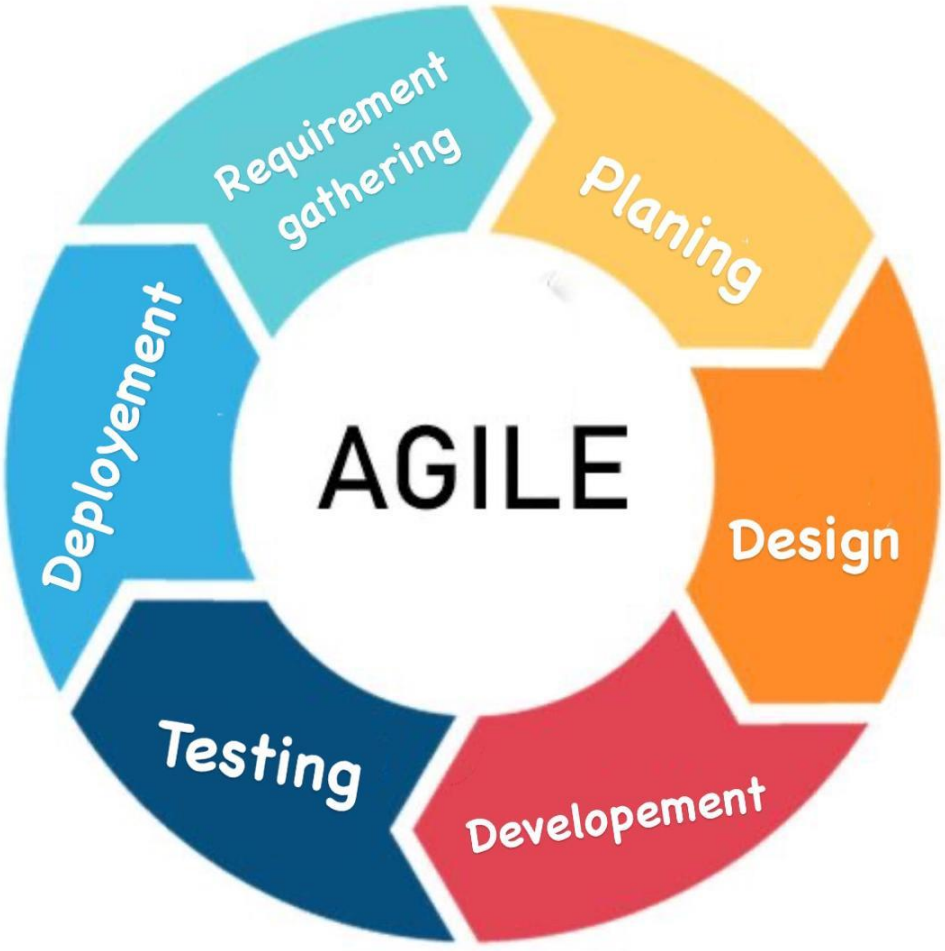
DESCRIPTIONS:

Phases	Description
1. Requirement Gathering	This arrange includes gathering nitty gritty input from partners to recognize the necessities of distribution center administration computer program. It clarifies anticipated highlights, essential information (things, amounts, costs, clients), and any imperatives or security concerns. The result could be a clear, significant list of necessities to direct arranging and plan.(Sommerville, 2016; Pressman and Maxim, 2014).

2. Planning	Amid arranging, the group breaks work into sprints, each with a particular objective like building a login framework. Errands are prioritized, assets allotted, and timelines evaluated. This guarantees organized advancement, practical objectives, and clear advance tracking's key for smooth communication and effective execution in Spry ventures (Agile Alliance, n.d.; VersionOne, 2020).
3. Design	The plan organize turns prerequisites into nitty gritty framework plans, counting UML charts, client interface formats, and database structures. It builds a establishment for improvement, guaranteeing smooth integration of components. A solid plan makes a difference designers get it frontend-backend interaction and guarantees useful and security needs are met (Sommerville, 2016; IEEE, 1998).
4. Development	Amid advancement, program is built in little increases through brief sprints. Engineers code highlights like login and item administration utilizing chosen dialects (e.g., JavaScript, Java). Frontend and backend are associated to the database. Dexterous advances visit testing and input, permitting the group to adjust and make strides the framework ceaselessly.
5. Testing	Testing guarantees each highlight works accurately, is user-friendly, and meets prerequisites. It happens amid and after each sprint, utilizing manual and computerized strategies to discover bugs, execution issues, and security imperfections. Input is rapidly tended to within the following cycle, keeping up quality and progressing unwavering quality and convenience (Sommerville, 2016; Agile Alliance, n.d.).

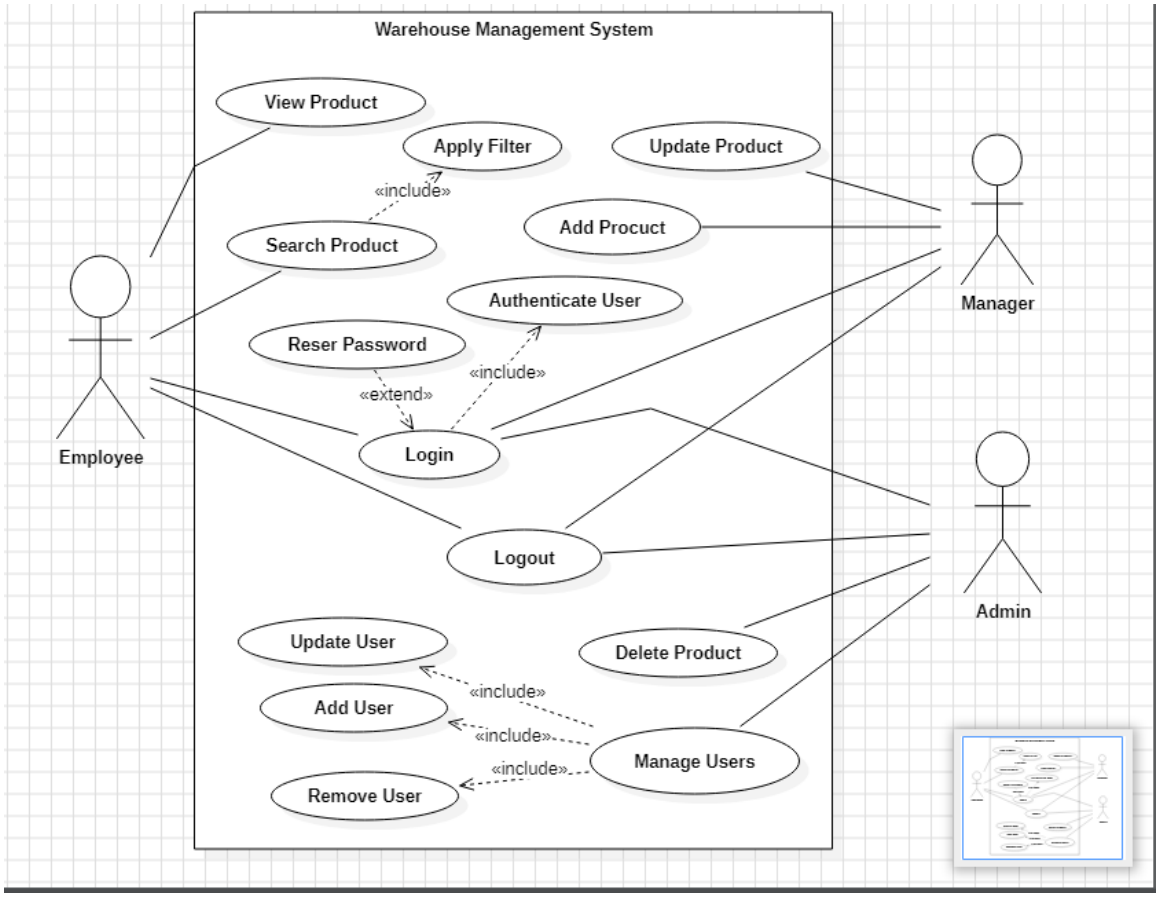
6. Deployment	Once a usable adaptation is tried and affirmed, it's conveyed to a live environment. This includes server setup, database setup, and guaranteeing the program runs easily for clients. In Spry, sending may happen after each sprint, empowering speedy conveyance, genuine client criticism, and ceaseless advancement within the following cycle.
7. Maintenance	Upkeep is an progressing organize where the framework is observed, upgraded, and progressed after sending. It includes settling bugs, improving highlights, optimizing execution, and adjusting to modern needs. In Spry, support is persistent and driven by client input, guaranteeing the stockroom framework remains useful, secure, and adjusted with advancing commerce requests.

AGILE SDLC LIFECYCLE:



UML Use Case Diagram

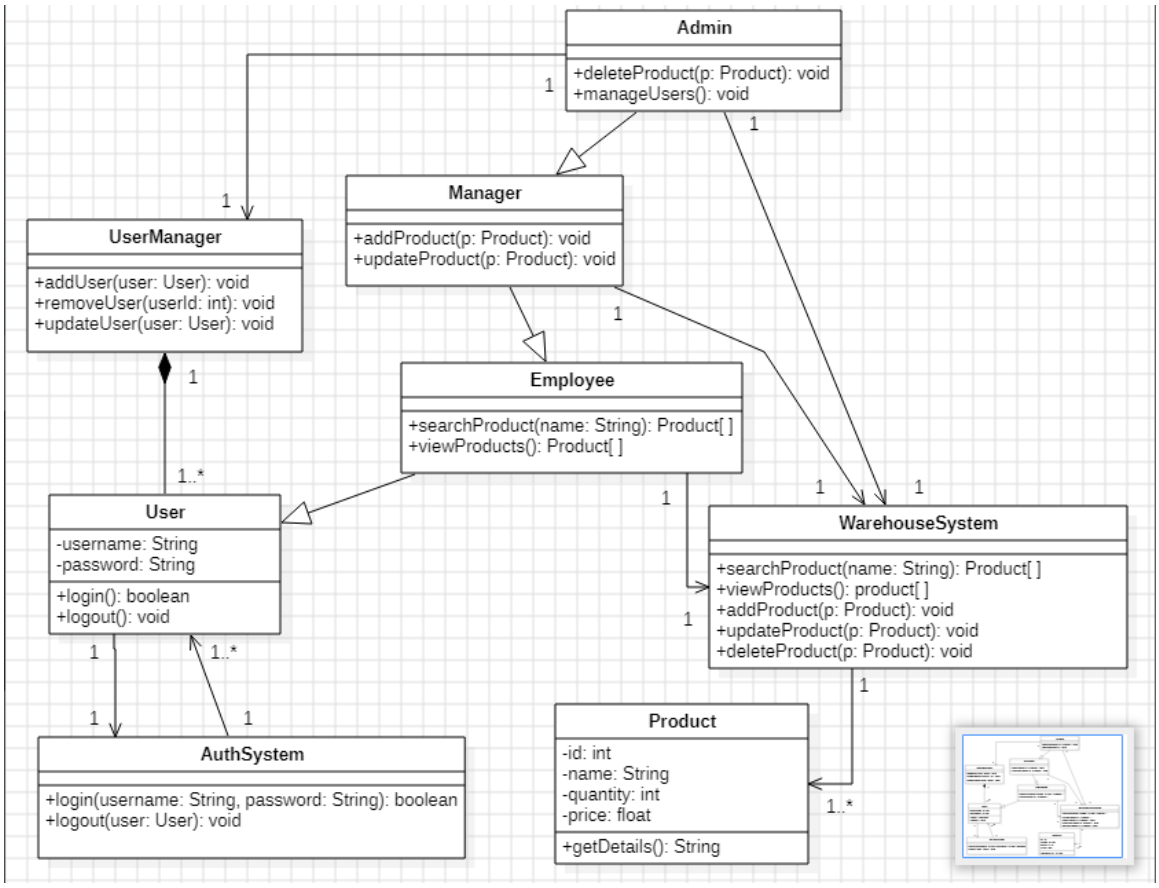
USE CASE DIAGRAM:



DESCRIPTION:

Admins oversee users and delete product. managers handle product stock. employees view and search product. Key utilize cases incorporate login/logout, product administration, user administration, and product look with channels.

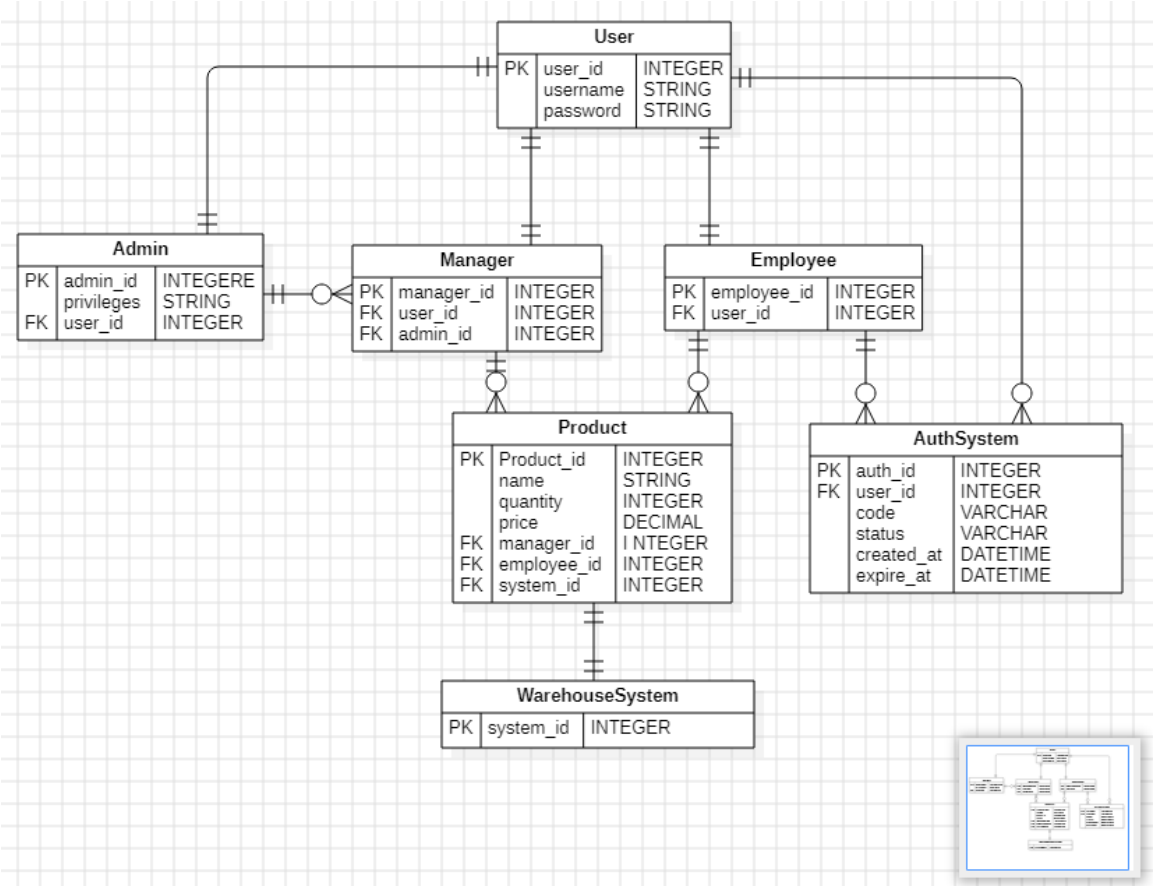
UML Class Diagram



DESCRIPTION:

Models core system objects ser, Product, AuthSystem, UserManager, and WarehouseSystem.. Characterizes parts (Admin, manager, employee) and their intelligent, exhibiting framework rationale, information structure, and user behavior taking care of.

UML Entity Relationship Diagram:



DESCRIPTION:

This ERD models a warehouse system with users classified as Admins, managers, or employees. Manager and employees oversee product. AuthSystem tracks user confirmation. product are linked to warehousesystem.

SIGN IN

- **All Users** (Admin, Manager, Employee)



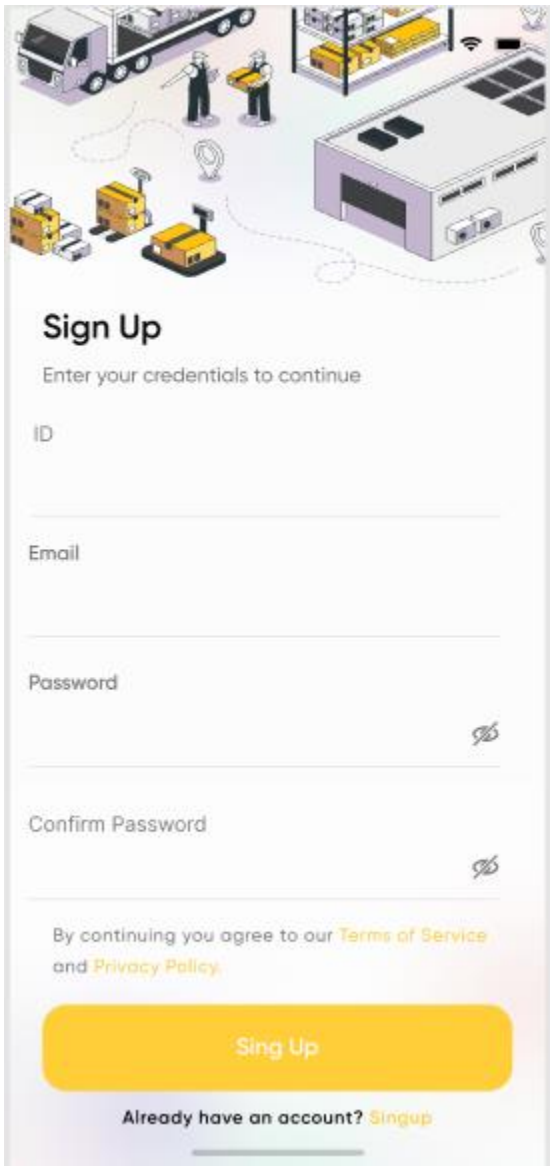
DESCRIPTION:

The Login Page serves as the section point for all users of the Warehouse Management System, counting Admin, Manager, and Employees. It permits users to safely verify themselves some time before accessing the system's functionalities.

SIGN UP

- **Accessed by all users:**

-



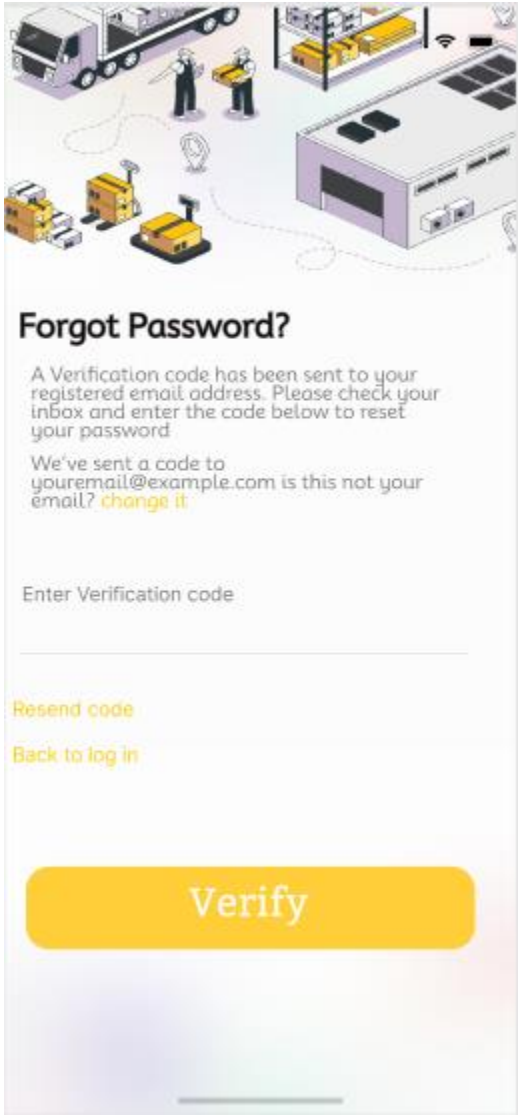
DESCRIPTION:

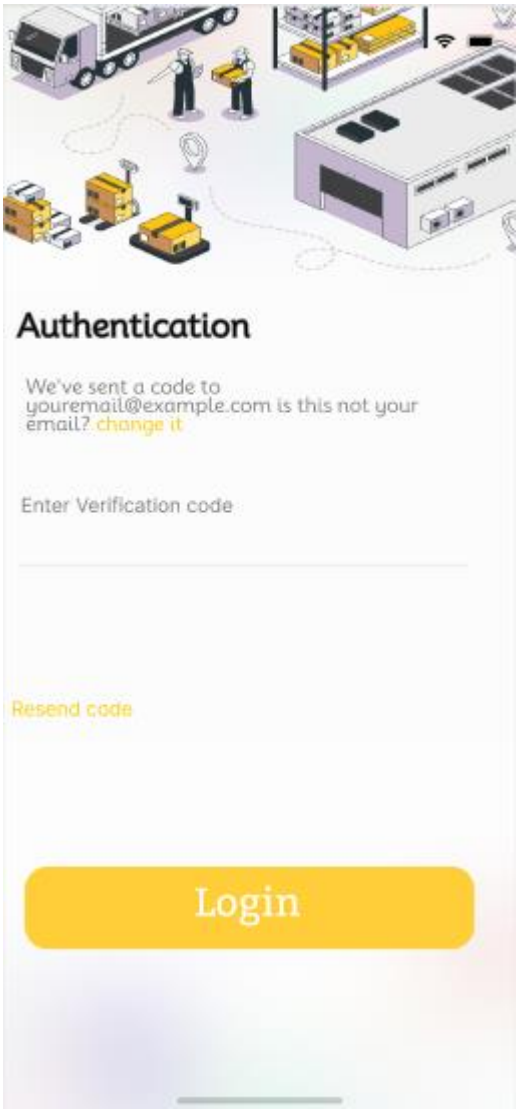
This page permits new Users to enroll for the Warehouse management system

Clients must enter their **ID** — which compares to the proficient card given to each distribution center staff part. Based on this ID, the framework consequently allocates the fitting part (admin, manager, or employee).

The shape also requires **E-mail**, **password**, and **confirm password** areas to guarantee account security.

FORGOT PASSWORD & AUTHENTICATION:





Forgot Password description:

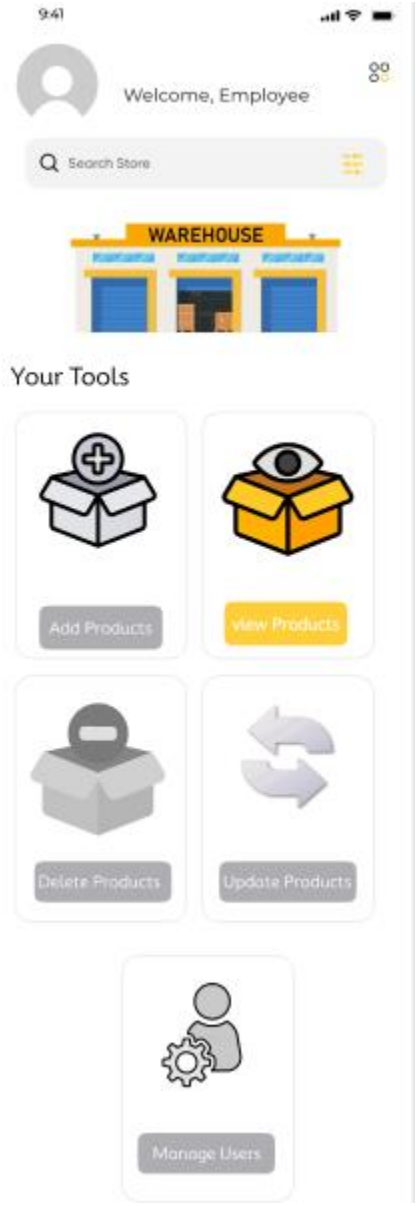
This page helps users reset their password by verifying their identity via email. After requesting a password reset, with the option to change the email if needed.

Authentication description:

This page is utilized to confirm the user's personality by entering a confirmation code sent to their enrolled mail.

Clients enter the confirmation code into the input field. On the off chance that they don't get the code, they can tap the **Resend** connect to urge a modern one.

EMPLOYEE HOME PAGE:

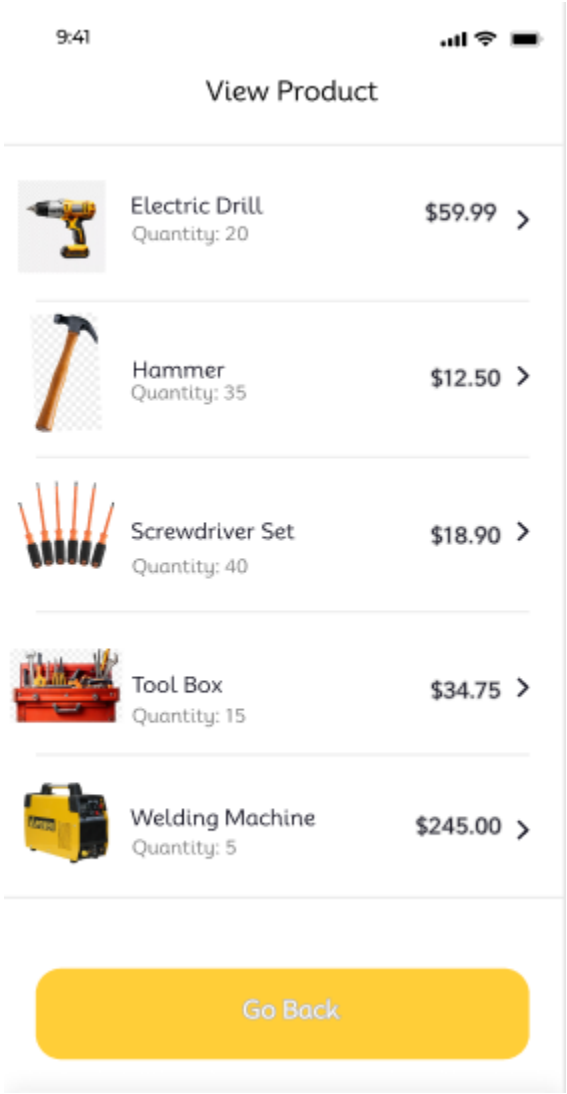


Home Page Description:

When an Employee logs in, they are directed to their main dashboard, designed to help them efficiently access product information.

Employees can view product details but cannot modify, add, or delete any items. The buttons or tools for those restricted actions appear grayed out and inactive, clearly indicating the limitations of their role.

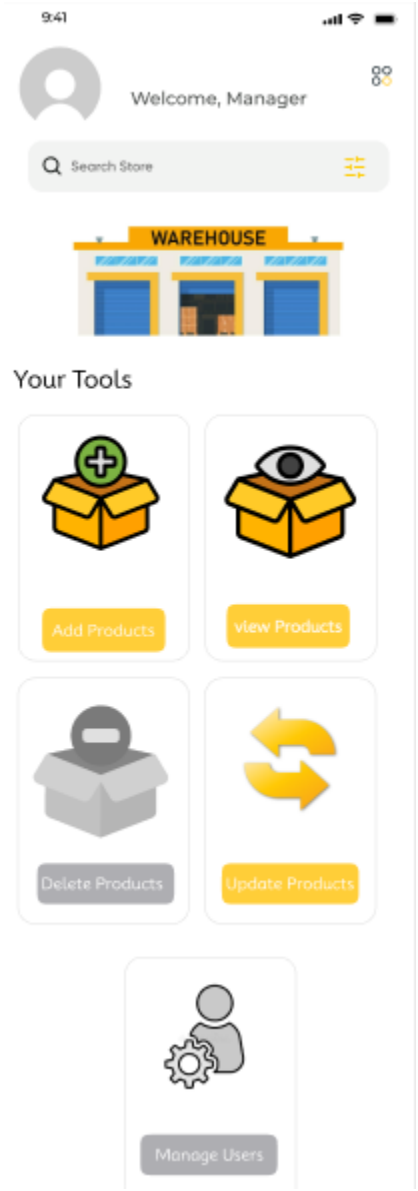
VIEW PRODUCTS:



View Products Description:

When a worker chooses to view items, they are presented with a detailed list or grid of products available within the warehouse.

MANAGER'S HOME PAGE:

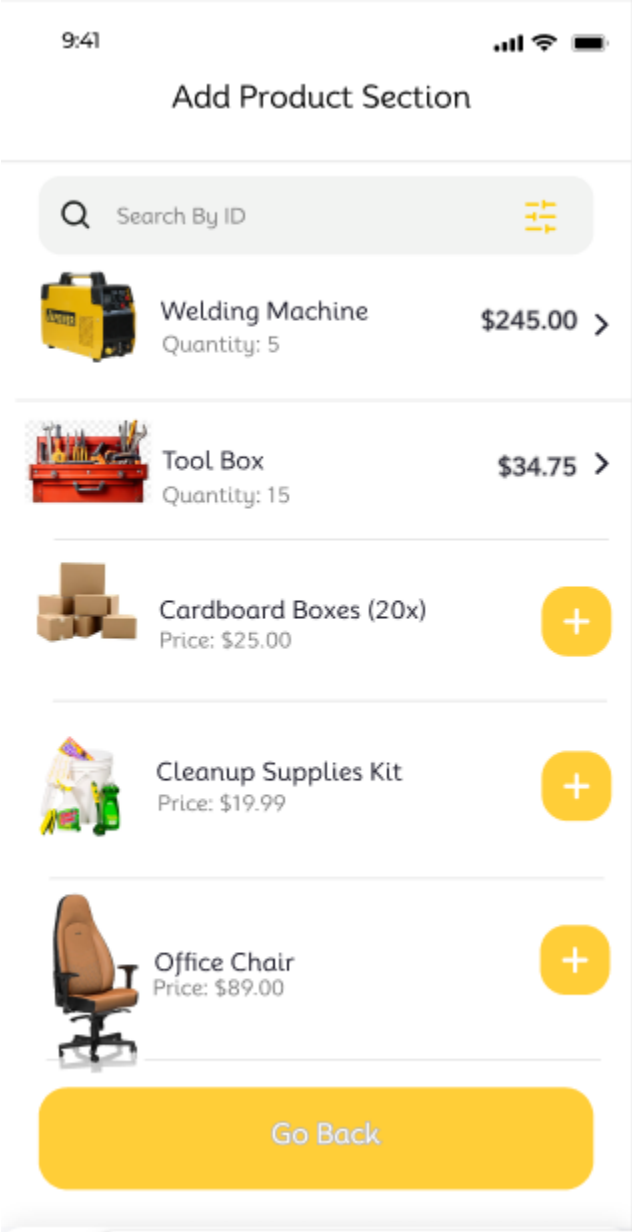


Home Page Description:

When a Manager logs in, they are taken to their dashboard which gives access to fundamental product Management tools. The Manager can:

- View items with detailed information
- Add new items to the warehouse stock
- Update existing item details

Add Product:

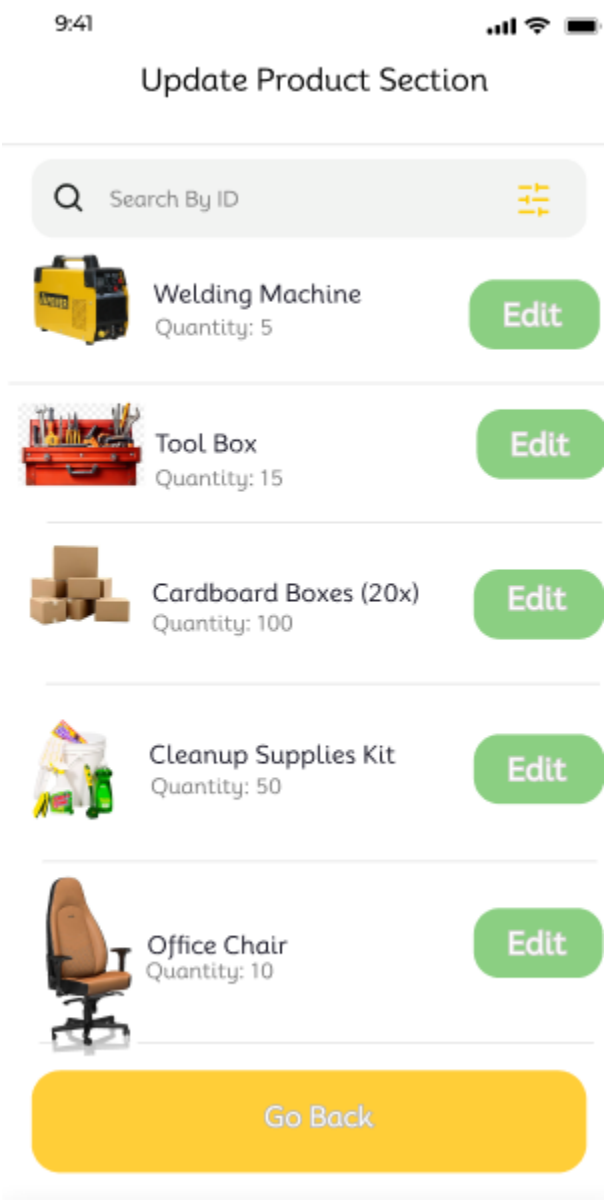


Add Products Description:

When a Manager chooses to include an item, the page shows two primary areas side-by-side or stacked for simple comparison:

- A list of items that as of now exist within the stockroom stock, making a difference to dodge copies.
- A list of unused items accessible to include to the System.

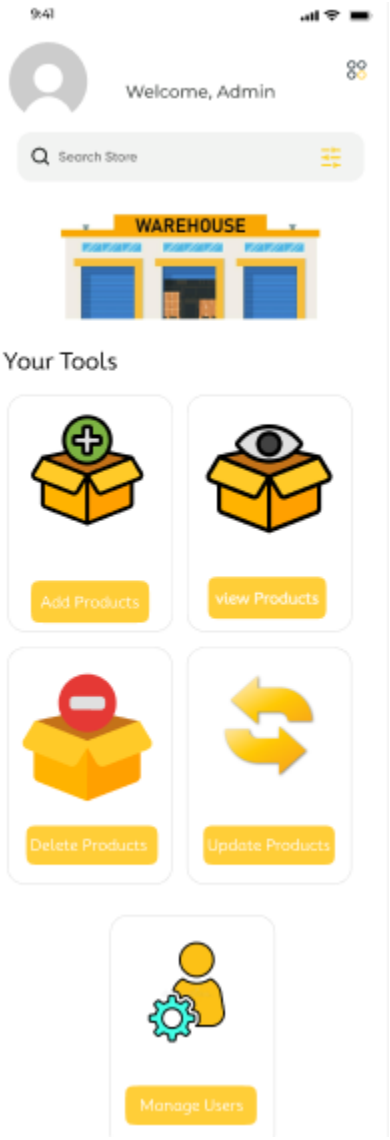
Update Page :



Update Page Description:

The update item page presents a list of all items as of now within the distribution center. Each item passage has an **Edit** button on the correct side, permitting the manager to alter item subtle elements such as title, cost, amount, and other significant data.

ADMIN HOME PAGE:



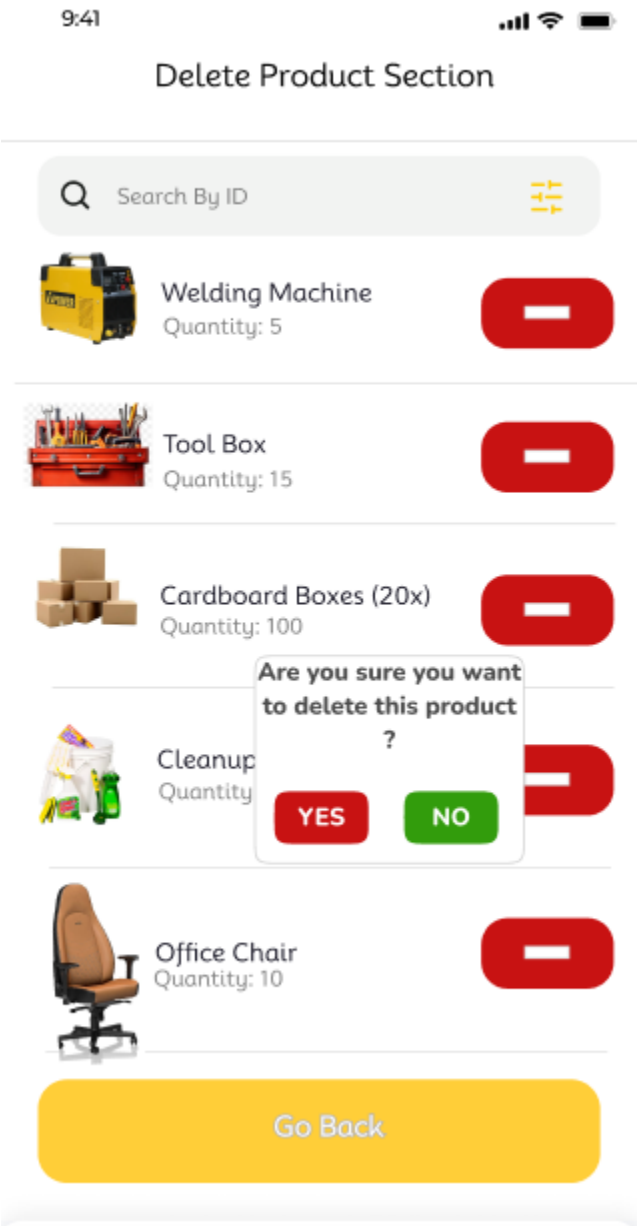
Home page Description:

This page is the main workspace for the Admin after logging in. It gives full access to all administration tools and features within the warehouse management system. Admins can:

- Add Items
- View Items
- Delete Items
- Update Items

- Manage Clients (including employee and manager accounts)

Delete Product:



Delete Page Description:

When an Admin chooses to erase an item, the page shows a list of all items within the warehouse. Each item incorporates a ruddy short symbol on the proper side showing the erase alternative.

Manage Users:





Manage Users Page Description:

This page permits the Admin to oversee all clients inside the distribution center administration framework. It shows a list of clients appearing their ID, Title, and Part (employee, manager, or Admin).

The admin can:

- Select and remove/delete any client from the framework.
- Update client data as required.

- Add modern clients by entering a modern user's ID, Title, and doling out a Part.

Backend database:

AUTH SYSTEM DATABASE:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	auth_id	int			No	None		AUTO_INCREMENT	Change Drop More
2	User_Id	int			No	None			Change Drop More
3	code	varchar(20)	utf8mb4_0900_ai_ci		No	None			Change Drop More
4	status	varchar(20)	utf8mb4_0900_ai_ci		No	None			Change Drop More
5	created_at	datetime			No	None			Change Drop More
6	expires_at	datetime			No	None			Change Drop More

The authsystem table oversees verification codes. It employments auth_id as the primary key and joins to user_id as a foreign key. It stores a created code, its status, creation time (created_at), and termination time (expired_at), set 5 minutes after creation to constrain legitimacy.

USER DATABASE:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	User_Id	int			No	None		AUTO_INCREMENT	Change Drop More
2	Username	varchar(100)	utf8mb4_0900_ai_ci		No	None			Change Drop More
3	Password	varchar(100)	utf8mb4_0900_ai_ci		No	None			Change Drop More

The client table stores fundamental account data. It employments user_id as the essential key to extraordinarily recognize each client. The table incorporates username for the user's login title and password to safely store the user's scrambled watchword, empowering verification and get to control.

MANAGER DATABASE:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	manager_id	int			No	None		AUTO_INCREMENT	Change Drop More
2	User_Id	int			No	None			Change Drop More
3	admin_id	int			No	None			Change Drop More

The manager table recognizes directors with manager_id as the primary key. It joins each manager to the user table by means of user_id and to the admin table through admin_id as foreign keys, building up connections that interface supervisors to their client accounts and related admins.

EMPLOYEE DATABASE:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	employee_id	int			No	None		AUTO_INCREMENT	Change Drop More
2	User_Id	int			No	None			Change Drop More

The representative table employments employee_id as theprimary key to interestingly distinguish each representative. It joins to the user table through the user_id foreign key, interfacing worker records to their comparing client accounts for confirmation and part administration.

ADMIN DATABASE:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	admin_id	int			No	None		AUTO_INCREMENT	Change Drop More
2	privilages	varchar(255)	utf8mb4_0900_ai_ci		No	None			Change Drop More
3	User_Id	int			No	None			Change Drop More

The admin table employments admin_id as the primary key to extraordinarily recognize each admin. It joins to the user table through user_id as a foreign key and incorporates a benefits field that characterizes the admin's get to rights and authorizations inside the framework.

PRODUCT DATABASE:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	product_id	int			No	None		AUTO_INCREMENT	Change Drop More
2	name	varchar(100)	utf8mb4_0900_ai_ci		No	None			Change Drop More
3	quantity	int			No	None			Change Drop More
4	price	int			No	None			Change Drop More
5	manager_id	int			No	None			Change Drop More
6	employee_id	int			No	None			Change Drop More
7	system_id	int			No	None			Change Drop More

The product table employments product_id as the primary key to extraordinarily recognize each item. It stores the product's name, quantity, and price. It moreover incorporates foreign keys connecting to manager_id, employee_id, and system_id, partner items with their dependable staff and framework records.

WAREHOUSE SYSTEM DATABASE:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	system_id	int			No	None		AUTO_INCREMENT	Change Drop More

The warehousesystem table contains one column, system_id, as the primary key. It identifies each warehouse system entry, acting as a central reference point for linking other related tables and managing the overall warehouse system data.

Backend application:

LOGIN:

```php
<?php
session_start();
require_once('../config/db.php'); // connect to database

// submission form checking
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $user_id = $_POST['user_id'] ?? '';
    $password = $_POST['password'] ?? '';

    if (empty($user_id) || empty($password)) {
        die('Please enter both User ID and Password.');
    }

    // check if the user exist or is a new user
    $stmt = $pdo->prepare("SELECT * FROM user WHERE user_id = ?");
    $stmt->execute([$user_id]);
    $user = $stmt->fetch();

    if ($user && password_verify($password, $user['password'])) {
        $_SESSION['user_id'] = $user['user_id'];

        //role detected by id
        $role = 'Unknown';


        $stmt = $pdo->prepare("SELECT * FROM admin WHERE user_id = ?");
        $stmt->execute([$user_id]);
        if ($stmt->fetch()) {
            $role = 'admin';
        }
```

```php
    $stmt = $pdo->prepare("SELECT * FROM manager WHERE user_id = ?");
    $stmt->execute([$user_id]);
    if ($stmt->fetch()) {
        $role = 'manager';
    }


    $stmt = $pdo->prepare("SELECT * FROM employee WHERE user_id = ?");
    $stmt->execute([$user_id]);
    if ($stmt->fetch()) {
        $role = 'employee';
    }

    $_SESSION['role'] = $role;

    // lead to the page based on the id and the role
    switch ($role) {
        case 'admin':
            header("Location: ../admin_dashboard.php");
            break;
        case 'manager':
            header("Location: ../manager_dashboard.php");
            break;
        case 'employee':
            header("Location: ../employee_dashboard.php");
            break;
        default:
            echo "Role not recognized.";
            break;
    }
```

```php
} else {
    echo "Invalid ID or password.";
}
```

Handle users login by confirming the given users ID and password against the database. It begins a session, sets users part data, and diverts users to their particular dashboards (admin, manager, employee). Too bolsters two-factor verification with confirmation codes.

LOGOUT:

```php
<?php
session_start();

$_SESSION = [];

if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();
    setcookie(session_name(), '', time() - 42000,
        $params["path"], $params["domain"],
        $params["secure"], $params["httponly"]
    );
}

session_destroy();

header("Location:loging.php");
exit;
?>
```

Crushes the current session to log the user out safely. After clearing all session information, it diverts the client back to the login page, guaranteeing no touchy information remains and avoiding unauthorized get to.

SIGNUP:

```php
<?php
require_once('../config/db.php');

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $user_id = $_POST['user_id'] ?? '';
    $email = $_POST['email'] ?? '';
    $password = $_POST['password'] ?? '';
    $confirm_password = $_POST['confirm_password'] ?? '';
    $role = $_POST['role'] ?? '';

    // can't keep the fields empty
    if (empty($user_id) || empty($email) || empty($password) || empty($confirm_password) || empty($role)) {
        die("All fields are required.");
    }

    // password match confirm password
    if ($password !== $confirm_password) {
        die("Passwords do not match.");
    }

    // role selection
    $allowed_roles = ['admin', 'manager', 'employee'];
    if (!in_array($role, $allowed_roles)) {
        die("Invalid role selected.");
    }

    // check if the user already exist
    $stmt = $pdo->prepare("SELECT * FROM user WHERE user_id = ? OR email = ?");
    $stmt->execute([$user_id, $email]);
    if ($stmt->fetch()) {
        die("User ID or email already exists.");
    }
```

```php
$hashed_password = password_hash($password, PASSWORD_DEFAULT);


$stmt = $pdo->prepare("INSERT INTO user (user_id, email, password) VALUES (?, ?, ?)");
$stmt->execute([$user_id, $email, $hashed_password]);


$stmt = $pdo->prepare("INSERT INTO $role (user_id) VALUES (?)");
$stmt->execute([$user_id]);

echo "Registration successful!";
```

Forms modern user enlistments by approving inputs like client ID, e-mail, and password. It safely hashes the password some time recently putting away it. Upon fruitful signup, the client is assigned a default part (usually employee) and spared within the database.

FORGOT PASSWORD:

```php
<?php
session_start();
require_once('../config/db.php');

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $email = $_POST['email'] ?? '';

    if (empty($email)) {
        die("Please enter your email.");
    }


    $stmt = $pdo->prepare("SELECT * FROM user WHERE email = ?");
    $stmt->execute([$email]);
    $user = $stmt->fetch();

    if ($user) {
        $_SESSION['user_id'] = $user['user_id'];

        // this part generate a verification code and store it
        $generated_code = rand(100000, 999999);
        $_SESSION['verification_code'] = $generated_code;
        $_SESSION['verify_action'] = 'reset_password';


        mail($user['email'], "Password Reset Code", "Your code is: $generated_code");

        echo "verification code (for testing): $generate_code";
        header("Location:../verify-code.php");
        exit;
    } else {
```

```php
} else {
    echo "No account found with that email.";
}
```

Permits users who forgot their password to ask a reset by submitting their enlisted mail. Sends a confirmation code by means of e-mail, which the user must enter to demonstrate character some time recently setting a new password.

VERIFY CODE:

```php
<?php
session_start();
require_once('../config/db.php');

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $user_id = $_POST['user_id'] ?? '';
    $password = $_POST['password'] ?? '';

    if (empty($user_id) || empty($password)) {
        die('Please enter both User ID and Password.');
    }


    $stmt = $pdo->prepare("SELECT * FROM user WHERE user_id = ?");
    $stmt->execute([$user_id]);
    $user = $stmt->fetch();

    if ($user && password_verify($password, $user['password'])) {
        $_SESSION['user_id'] = $user['user_id'];


        $role = 'Unknown';

        $stmt = $pdo->prepare("SELECT * FROM admin WHERE user_id = ?");
        $stmt->execute([$user_id]);
        if ($stmt->fetch()) $role = 'admin';

        $stmt = $pdo->prepare("SELECT * FROM manager WHERE user_id = ?");
        $stmt->execute([$user_id]);
        if ($stmt->fetch()) $role = 'manager';

        $stmt = $pdo->prepare("SELECT * FROM employee WHERE user_id = ?");
```

```
        $stmt = $pdo->prepare("SELECT * FROM employee WHERE user_id = ?");
        $stmt->execute([$user_id]);
        if ($stmt->fetch()) $role = 'employee';

        $_SESSION['role'] = $role;


        $generated_code = rand(100000, 999999);
        $_SESSION['verification_code'] = $generated_code;
        $_SESSION['verify_action'] = 'login';

        // Send the code to the email
        mail($user['email'], "Login Verification Code", "Your verification code is: $generated_code");


        exit;
    } else {
        echo "Invalid ID or password.";
    }
}
?>
```

Handles confirmation of codes sent for both login confirmation and password reset. Checks if that the submitted code matches the one put away within the session, at that point then either grants access or permits password change, depending on the action context.

DATABASE LINK:

```
<?php
$host = 'localhost';
$db   = 'warehouse_management';
$user = 'root';
$pass = 'Reem1234@';
$charset = 'utf8mb4';

$dsn = "mysql:host=$host;dbname=$db;charset=$charset";
$options = [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
];

try {
    $pdo = new PDO($dsn, $user, $pass, $options);
} catch (\PDOException $e) {
    die('Database connection failed: ' . $e->getMessage());
}
?>
```

This record sets up and oversees the association between the PHP file and the MySQL database utilizing PDO. It sets database qualifications, handles association mistakes, and gives a reusable $pdo question for secure, arranged SQL inquiries all through the application, guaranteeing proficient and secure database intuitive.

ADD PRODUCT, ADD PRODUCT HANDLER:

```php
<?php
session_start();
require_once('../config/db.php');


if (!isset($_SESSION['user_id']) || !in_array($_SESSION['role'], ['admin', 'manager'])) {
    die("Access denied.");
}


$stmt = $pdo->query("SELECT * FROM product");
$existing_products = $stmt->fetchAll();


?>
```

```php
<?php
session_start();
require_once('../config/db.php');

// only admin and manager who are responsible
if (!isset($_SESSION['user_id']) || !in_array($_SESSION['role'], ['admin', 'manager'])) {
    die("Access denied.");
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $name = $_POST['name'] ?? '';
    $quantity = $_POST['quantity'] ?? '';
    $price = $_POST['price'] ?? '';

    if (empty($name) || empty($quantity) || empty($price)) {
        die("All fields are required.");
    }

    // add new product
    $stmt = $pdo->prepare("INSERT INTO product (name, quantity, price) VALUES (?, ?, ?)");
    $stmt->execute([$name, $quantity, $price]);


    header("Location: add_product.php");
    exit;
}
?>
```

The add product page appears existing items and lets authorized users enter new ones. The handler process the enter demands, approves inputs, and embeds new products records into the database, restricting access to managers and admins only.

DELETE PRODUCT:

```php
<?php
session_start();
require_once('../config/db.php');


if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'admin') {
    die("Access denied.");
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $product_id = $_POST['product_id'] ?? '';

    if (empty($product_id)) {
        die("No product selected.");
    }

    // Delete the product
    $stmt = $pdo->prepare("DELETE FROM product WHERE product_id = ?");
    $stmt->execute([$product_id]);


    header("Location: list_products_for_delete.php");
    exit;
}
?>
```

```php
<?php
session_start();
require_once('../config/db.php');


if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'admin') {
    die("Access denied.");
}


$stmt = $pdo->query("SELECT * FROM product");
$products = $stmt->fetchAll();
?>
```

The list product for deletion shows all items with delete buttons. When a client clicks delete, the delete product handler affirms the activity (e.g., through popup) and expels the item from the database. As it were admins have this consent.

UPDATE PRODUCT, UPDATE PRODUCT HANDLER:

```php
<?php
session_start();
require_once('../config/db.php');


if (!isset($_SESSION['user_id']) || !in_array($_SESSION['role'], ['admin', 'manager'])) {
    die("Access denied.");
}


$stmt = $pdo->query("SELECT * FROM product");
$products = $stmt->fetchAll();


?>
```

```php
<?php
session_start();
require_once('../config/db.php');


if (!isset($_SESSION['user_id']) || !in_array($_SESSION['role'], ['admin', 'manager'])) {
    die("Access denied.");
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $product_id = $_POST['id'] ?? null;
    $name = $_POST['name'] ?? '';
    $quantity = $_POST['quantity'] ?? '';
    $price = $_POST['price'] ?? '';

    if (!$product_id || empty($name) || empty($quantity) || empty($price)) {
        die("All fields are required.");
    }


    $stmt = $pdo->prepare("UPDATE product SET name = ?, quantity = ?, price = ? WHERE id = ?");
    $stmt->execute([$name, $quantity, $price, $product_id]);


    header("Location: update_product.php");
    exit;
}
?>
```

The upgrade product page records products with edit buttons. Clicking edit opens a shape to edit product subtle elements. The overhaul handler approves changes and upgrades the database. Available as it were by managers and admins.

VIEW PRODUCT:

```php
<?php
session_start();
require_once('../config/db.php');


if (!isset($_SESSION['user_id'])) {
    die("Access denied.");
}


$stmt = $pdo->query("SELECT * FROM product");
$products = $stmt->fetchAll();


?>
```

Shows the complete product list for all users (admins, manager, employee) to browse. No altering or cancellation rights here, fair read-only get to product information.

MANAGE USERS:

```php
<?php
session_start();
require_once('../config/db.php');


if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'admin') {
    die("Access denied.");
}

$stmt = $pdo->query("SELECT * FROM user");
$users = $stmt->fetchAll();


?>
```

Shows a list of all users with choices to add, edit, or delete users. As it were open by admins, this page gives the interface to oversee user accounts, appearing user subtle elements and activity buttons that trigger the particular handlers for overhauling or evacuating users.

ADD USER HANDLER:

```php
<?php
session_start();
require_once('../config/db.php');

if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'admin') {
    die("Access denied.");
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $user_id = $_POST['user_id'] ?? '';
    $email = $_POST['email'] ?? '';
    $password = $_POST['password'] ?? '';
    $role = $_POST['role'] ?? 'employee'; // default role can be changed by selection

    if (empty($user_id) || empty($email) || empty($password)) {
        die("All fields are required.");
    }


    $hashed_password = password_hash($password, PASSWORD_DEFAULT);


    $stmt = $pdo->prepare("INSERT INTO user (user_id, email, password) VALUES (?, ?, ?)");
    $stmt->execute([$user_id, $email, $hashed_password]);


    if ($role === 'admin') {
        $stmt = $pdo->prepare("INSERT INTO admin (user_id) VALUES (?)");
        $stmt->execute([$user_id]);
    } elseif ($role === 'manager') {
        $stmt = $pdo->prepare("INSERT INTO manager (user_id) VALUES (?)");
        $stmt->execute([$user_id]);
```

Forms the frame accommodation for including a new user. It approves the input, safely hashes the password, assigns a role, and inserts the new users record into the database. limited to admins to preserve control over user creation.

DELETE USER:

```php
<?php
session_start();
require_once('../config/db.php');

if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'admin') {
    die("Access denied.");
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $user_ids = $_POST['user_ids'] ?? [];

    if (empty($user_ids) || !is_array($user_ids)) {
        die("No users selected.");
    }

    foreach ($user_ids as $user_id) {
        // Delete from all role tables
        $pdo->prepare("DELETE FROM admin WHERE user_id = ?")->execute([$user_id]);
        $pdo->prepare("DELETE FROM manager WHERE user_id = ?")->execute([$user_id]);
        $pdo->prepare("DELETE FROM employee WHERE user_id = ?")->execute([$user_id]);

        // Delete from user table
        $pdo->prepare("DELETE FROM user WHERE user_id = ?")->execute([$user_id]);
    }

    header("Location: manage_users.php");
    exit;
}
?>
```

Handles deletion demands for users. Upon accepting a user ID, it confirms admin authorizations and evacuates the comparing user from the database safely. This anticipates unauthorized erasures and guarantees as it were admins can remove user's accounts.

UPDATE USER:

```php
<?php
session_start();
require_once('../config/db.php');

if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'admin') {
    die("Access denied.");
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $user_id = $_POST['user_id'] ?? '';
    $email = $_POST['email'] ?? '';
    $password = $_POST['password'] ?? '';
    $role = $_POST['role'] ?? '';

    if (empty($user_id) || empty($email) || empty($role)) {
        die("User ID, email, and role are required.");
    }

    if (!empty($password)) {
        $hashed_password = password_hash($password, PASSWORD_DEFAULT);
        $stmt = $pdo->prepare("UPDATE user SET email = ?, password = ? WHERE user_id = ?");
        $stmt->execute([$email, $hashed_password, $user_id]);
    } else {
        $stmt = $pdo->prepare("UPDATE user SET email = ? WHERE user_id = ?");
        $stmt->execute([$email, $user_id]);
    }

    // Update role, delete old and add new
    $pdo->prepare("DELETE FROM admin WHERE user_id = ?")->execute([$user_id]);
    $pdo->prepare("DELETE FROM manager WHERE user_id = ?")->execute([$user_id]);
    $pdo->prepare("DELETE FROM employee WHERE user_id = ?")->execute([$user_id]);
```

Gets and forms upgrades to client data, such as email, password, or part changes. It approves inputs, applies changes within the database, and affirms the update was effective. As it were admins can perform these update to preserve user's data integrity.

ADMIN DASHBOARD:

```php
<?php
session_start();
require_once('config/db.php');


if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'admin') {
    die("Access denied.");
}


$stmt = $pdo->query("SELECT COUNT(*) FROM user");
$total_users = $stmt->fetchColumn();


$stmt = $pdo->query("SELECT COUNT(*) FROM product");
$total_products = $stmt->fetchColumn();


?>
```

Gives an outline of the framework for admins, counting speedy links to manage user, products, and system settings. It may show key stats like user checks, product stock status, and recent activities. entirely for admins to manage and control warehouse's operations.

MANAGER DASHBOARD:

```php
<?php
session_start();

if (isset($_SESSION['user_id'])) {

    switch ($_SESSION['role']) {
        case 'admin':
            header('Location: admin_dashboard.php');
            break;
        case 'manager':
            header('Location: manager_dashboard.php');
            break;
        case 'employee':
            header('Location: employee_dashboard.php');
            break;
        default:
            // Unknown role, log out or show error
            session_destroy();
            header('Location: auth/login.php');
            break;
    }
} else {
    // Not logged in, go to login page
    header('Location: auth/login.php');
}
exit;
```

Offers managers a summary of product stock, pending tasks, and group exercises. It permits to get to product administration highlights but confines user administration. This dashboard makes a difference directors screen day by day operations and make educated choices inside their scope.

EMPLOYEE DASHBOARD:

```php
<?php
session_start();
require_once('config/db.php');

if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'employee') {
    die("Access denied.");
}


?>
```

Shows basic information for workers such as assigned tasks, product seeing capabilities, and basic system notices. Limited access, primarily read-only, centering on their operational obligations without consents to add, delete, or update records.

INDEX PAGE:

```php
<?php
session_start();

if (isset($_SESSION['user_id'])) {

    switch ($_SESSION['role']) {
        case 'admin':
            header('Location: admin_dashboard.php');
            break;
        case 'manager':
            header('Location: manager_dashboard.php');
            break;
        case 'employee':
            header('Location: employee_dashboard.php');
            break;
        default:
            // Unknown role, log out or show error
            session_destroy();
            header('Location: auth/login.php');
            break;
    }
} else {
    // Not logged in, go to login page
    header('Location: auth/login.php');
}
exit;
```

Acts as the passage point of the app, checking user session and part. It diverts users to their particular dashboards based on their part or to the login page in the event that not confirmed, guaranteeing legitimate get to control and route stream right after login.

Appendices

Agile:

pdf1.pdf

Usecase diagram

pdf2.pdf

Uml cass diagram

pdf3.pdf

Uml entitie relationship

pdf3.pdf

Front end design

pdf 4.pdf

Database

pdf5.pdf

Backend

warehouse-system.zip

References

Agile Alliance (n.d.) *Agile Glossary*. Available at: https://www.agilealliance.org (Accessed: 16 May 2025).

IEEE (1998) *IEEE Standard for Developing Software Life Cycle Processes (IEEE Std 1074-1997)*. New York: IEEE. Available at: https://standards.ieee.org/ieee/1074/1606/ (Accessed: 16 May 2025).

Pressman, R.S. and Maxim, B.R. (2014) *Software Engineering: A Practitioner's Approach*. 8th edn. New York: McGraw-Hill Education.

Sommerville, I. (2016) *Software Engineering*. 10th edn. Harlow: Pearson Education.

VersionOne (2020) *14th Annual State of Agile Report*. Available at: https://stateofagile.com (Accessed: 16 May 2025).

Oracle Corporation, 2025. *Oracle PHP Developer Center*. Available at: https://www.oracle.com/developer/php/ [Accessed 21 May 2025].